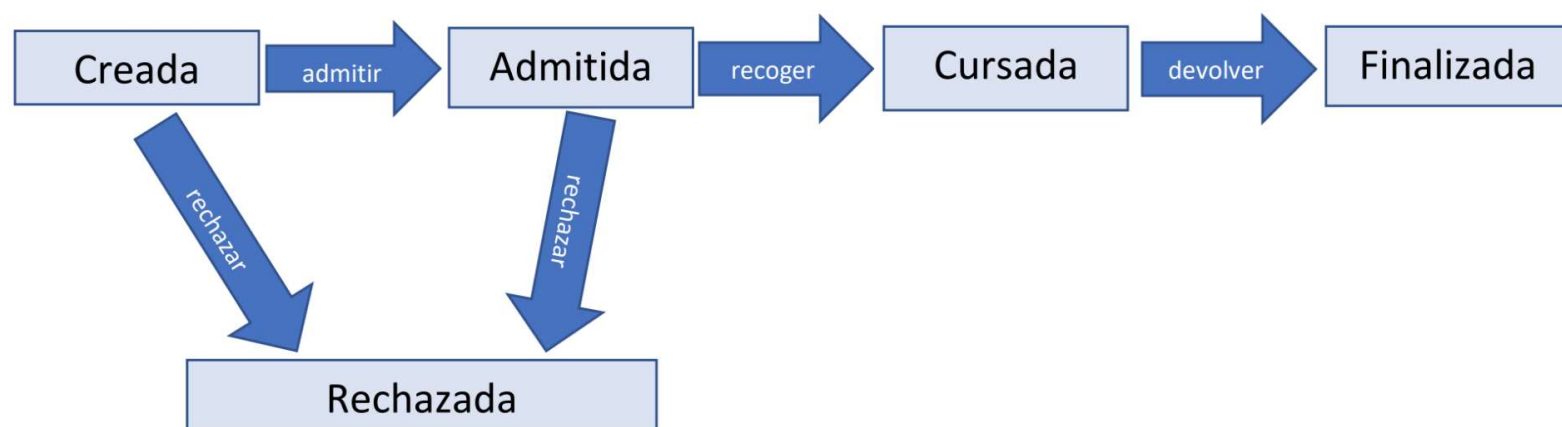


Ejercicio 31. State Pattern. Caso de uso

Supongamos que queremos diseñar el ciclo de vida de una **petición electrónica de reserva** de un libro de nuestra biblioteca que puede pasar por los siguientes estados:

- **Creada**, es el estado inicial de la petición, en esta se recogerán los siguientes datos para su procesamiento, o Tipo de usuario, (alumno, docente) o Libro solicitado (título, autor)
- **Admitida**, la petición ha sido admitida y queda pendiente de recogerse el libro de interés.
- **Cursada**, el libro se ha recogido para el disfrute del usuario.
- **Finalizada**, el libro ha sido devuelto.
- **Rechazada**, la petición de reserva ha sido rechazada.

Diagrama de estados de una petición de préstamo bibliotecario.



Que tiene las siguientes transiciones:

- **Admitir**, partiendo de los datos de la petición, se comprueba que el usuario tenga la cuenta activa para admitir la petición o rechazarla en caso contrario. Se enviará al usuario una notificación indicando la aprobación de su petición y recordando que tiene 2 días para hacer efectiva la recogida del libro. usuario
- **Recoger**, partiendo de una petición admitida, se comprueba que no se ha retrasado más del tiempo establecido para recogerlo (2 días), pasa a estado Cursada, Rechazada en caso contrario. Si se cumplen las condiciones de recogida, se enviará una notificación al usuario confirmando la recogida del libro y recordando los días disponibles del préstamo. Los docentes tienen 10 días de préstamo y los alumnos 15.
- **Devolver**, partiendo de una petición Cursada, se envía una notificación al usuario certificando su devolución y el estado en el que ha sido recepcionado el libro.

Se pide:

- Diseñar la solución utilizando todos los patrones identificados (State, Observer, Strategy, Singleton)
- Utilizando la clase Test, **LibraryLoanTest**, simular la reserva de dos préstamos, una para un profesor y otra para un estudiante, y el tratamiento de la petición en cada uno de los estados.

Clase test.

```
package com.utad.inso.diso.state.library;

import com.utad.inso.diso.observer.library.pullpush.Book;
import com.utad.inso.diso.observer.library.pullpush.BookState;

public class LibraryLoanTest {
    public static void main(String[] args) {
        LoansLibrary library = new LoansLibrary(); // Singleton
        Book bookGood = new Book("Gang of four Design patterns",
                                "Gang of four", BookState.GOOD);
        //TODO Los docentes tienen 10 días de préstamo, los alumnos 15 días.
        LibraryUser professor = new LibraryUser ("MA",
            "miguel.mesas@u-tad.com", UserType.PROFESSOR, true);
        LibraryUser student = new LibraryUser ("Inés",
            "ines@live.u-tad.com", UserType.STUDENT, true);

        LibraryLoanRequestContext professorLoanRequest
            = new LibraryLoanRequestContext(library, bookGood, professor);
        LibraryLoanRequestContext studentLoanRequest
            = new LibraryLoanRequestContext(library, bookGood, student);
        //La biblioteca gestiona las reservas recién creadas
        library.processLibraryLoan(studentLoanRequest);
        library.processLibraryLoan(professorLoanRequest);

        //La biblioteca gestiona la recogida del libro del docente.
        library.processLibraryLoan(professorLoanRequest);
        //La biblioteca gestiona la recogida del libro del alumno.
        library.processLibraryLoan(studentLoanRequest);

        //La biblioteca gestiona la devolución del libro del alumno.
        library.returnBook(studentLoanRequest.getBook(),
                           studentLoanRequest);
        //La biblioteca gestiona la devolución del libro del docente.
        library.returnBook(professorLoanRequest.getBook(),
                           professorLoanRequest);
    }
}
```

Resultados por consola:

```

Notifcation to user : Inés, library loan admited on 2023-03-27,
due date to pickup 2023-03-29
Notifcation to user : MA, library loan admited on 2023-03-27, due
date to pickup 2023-03-29
Notifcation to user : MA, library loan pickup on 2023-03-27, due
date to return 2023-04-06 days(10)
Notifcation to user : Inés, library loan pickup on 2023-03-27,
due date to return 2023-04-11 days(15)
Notifcation to user : Inés, library loan returned on
2023-03-27,Book [title=Gang of four Design patterns, author=Gang
of four, bookState=GOOD]
Notifcation to user : MA, library loan returned on
2023-03-27,Book [title=Gang of four Design patterns, author=Gang
of four, bookState=GOOD]

```

Notificaciones y estrategia.

Para las notificaciones a los usuarios, utilizaremos el patrón Observer a partir del modelo de notificación visto en ejercicios anteriores. En este caso, las diferentes formas de notificar serán una variante push, por ello debes considerar **PushNotificationStrategy** como punto de partida.

```

public class PushNotificationStrategy implements NotificationStrategy {

    public void update(ModelPullPushObserver modelObserver,
                      ModelPullPushObservable modelObservable,
                      Object object) {
        modelObserver.update(null, object);
    }
}

```

Tipo de usuario

Para el tipo de usuario, vamos a utilizar un tipo enumerado.

```

public enum UserType {
    STUDENT(?), PROFESSOR(?);
    private final int loanDays;
}

```

Usuario de biblioteca

El usuario de biblioteca tendrá al menos esta información

```
public class LibraryUser ¿??? Cómo decimos que es un usuario que va a
consumir notificaciones {
    private String name;
    private String address;
    private UserType userType;
    private Boolean activeAccount;
    public LibraryUser(String name, String address,
                        UserType userType, Boolean activeAccount) {
        super();
        this.name = name;
        this.address = address;
        this.userType = userType;
        this.activeAccount = activeAccount;

        //TODO Los usuarios no recibirán acceso a la biblioteca y las
        notificaciones estarán formadas por texto exclusivamente
    }
}
```

Biblioteca

Trabajaremos con una biblioteca especializada que gestiona los préstamos bibliotecarios además de la devolución de los libros.

```
package com.utad.inso.diso.state.library;

import com.utad.inso.diso.observer.library.pullpush.Book;
import com.utad.inso.diso.observer.library.pullpush.Library;

public class LoansLibrary extends Library {

    public LoansLibrary() {
        super();
    }

    public void returnBook(Book book,
                           LibraryLoanRequestContext libraryLoanRequestContext) {
        //TODO devolución y gestión del préstamo
    }

    public void processLibraryLoan(
        LibraryLoanRequestContext libraryLoanRequestContext) {
        //TODO ¿Quién debe procesar el préstamo bibliotecario?
    }
}
```

Préstamo bibliotecario

Diseño de LibraryLoanRequest (reserva de préstamo bibliotecario).

```
import java.time.LocalDate;
public class LibraryLoanRequestContext {
    private Tipo1???                currentState;
    private Tipo1???Tipo2???        x;
    private TipoX                    y;
    private TipoX                    z;
    //Necesitaremos un modelo observable sujeto a diferentes tipos de
    notificaciones
    ...
    private LoansLibrary             loansLibrary ;
    private Book                     book;
    private LibraryUser              libraryUser;
    private LocalDate                createdDate;
    private LocalDate                pickupDate;

    public void process() {
        //TODO ¿Quién debe procesar este método?
    }

    public LibraryLoanRequestContext(LoansLibrary loansLibrary ,
                                     Book book, LibraryUser libraryUser) {
        super();
        this.loansLibrary = loansLibrary;
        this.book = book;
        this.libraryUser = libraryUser;
        //TODO ¿Cómo indicamos que la reserva comienza en estado
        Creada?
    }
```

Estados

- ¿Qué interface utilizamos para los estados?
- ¿Y para las transiciones?
- ¿Qué relación existe entre la interface de servicio de estado y sus transiciones?

Diagrama UML simplificado

